Script de Conectividad Funcional fMRI

https://gemini.google.com/app/4abddfcdc25f3f4a

User prompt: me puedes dar un script para el reprocesamiento, filtrado de ruido para que haga un analisis de conectividad fmri

Response: Este es un script en **Python** estructurado paso a paso utilizando **Nilearn**, la librería estándar para el procesamiento de fMRI en Python. El flujo realiza las etapas esenciales para preparar señales BOLD antes de calcular la matriz de conectividad funcional.

Requisitos previos

Asegúrate de instalar las dependencias necesarias en tu entorno virtual:

```
pip install nilearn numpy matplotlib pandas
```

Script de procesamiento y conectividad

```
import numpy as np
import pandas as pd
from nilearn import datasets, input_data, connectome, plotting
import matplotlib.pyplot as plt

# ==============================================================================
# 1. CARGA DE DATOS (Ejemplo con dataset de Nilearn)
# ==============================================================================
# Reemplaza 'func_filename' y 'confounds_filename' con las rutas a tus archivos.
data = datasets.fetch_development_fmri(n_subjects=1)
func_filename = data.func[0]
confounds_filename = data.confounds[0]

# Parámetros del escáner
TR = 2.0  # Tiempo de Repetición en segundos (ajusta según tu secuencia)

# ==============================================================================
# 2. DEFINICIÓN DEL ATLAS / PARCELACIÓN
# ==============================================================================
# Usamos el atlas Schaefer (400 regiones). Puedes cambiarlo por Harvard-Oxford, AAL, etc.
atlas = datasets.fetch_atlas_schaefer_2018(n_rois=400, resolution_mm=2)
atlas_filename = atlas.maps
labels = atlas.labels

# ==============================================================================
# 3. EXTRACTION, FILTRADO Y LIMPIEZA DE LA SEÑAL (Nittering / Preprocesamiento)
# ==============================================================================
# NiftiLabelsMasker realiza el filtrado de ruido en un solo paso optimizado:
# - Filtrado pasabanda (Bandpass filtering): 0.01 - 0.1 Hz (estándar para rs-fMRI)
# - Regresión de confusos (movimiento, señal global, LCR, materia blanca)
# - Detrending y Estandarización (z-score)

masker = input_data.NiftiLabelsMasker(
    labels_img=atlas_filename,
    standardize=True,      # Z-score para normalizar la varianza
    detrend=True,          # Remueve tendencias lineales (drift del escáner)
    low_pass=0.1,          # Límite superior del filtro pasabanda (Hz)
    high_pass=0.01,        # Límite inferior del filtro pasabanda (Hz)
    t_r=TR,                # Requerido para el filtrado temporal
    memory='nilearn_cache',# Guarda en caché para acelerar ejecuciones futuras
    verbose=1
)

# Cargar los confusos (Head motion parameters, WM, CSF)
# Si usaste fMRIPrep, puedes seleccionar columnas específicas aquí.
confounds_df = pd.read_csv(confounds_filename, sep='\t')

# Extraer y filtrar la serie temporal de cada ROI
print("Extrayendo y filtrando series temporales...")
time_series = masker.fit_transform(func_filename, confounds=confounds_filename)

# ==============================================================================
# 4. ANÁLISIS DE CONECTIVIDAD FUNCIONAL
# ==============================================================================
# Cálculo de la matriz de correlación (Correlación de Pearson)
correlation_measure = connectome.ConnectivityMeasure(kind='correlation')
correlation_matrix = correlation_measure.fit_transform([time_series])[0]
```

```
# Limpiar la diagonal (autocorrelación de la región consigo misma = 1)
np.fill_diagonal(correlation_matrix, 0)

# ==============================================================================
# 5. VISUALIZACIÓN DE RESULTADOS
# ==============================================================================
# A. Visualizar la Matriz de Conectividad
plt.figure(figsize=(10, 8))
plt.imshow(correlation_matrix, cmap='coolwarm', vmin=-1, vmax=1)
plt.colorbar(label='Correlación de Pearson (r)')
plt.title('Matriz de Conectividad Funcional')
plt.xlabel('Regiones de Interés (ROIs)')
plt.ylabel('Regiones de Interés (ROIs)')
plt.show()

# B. Visualizar el Conectoma en 3D sobre el cerebro
coords = atlas.region_coords if hasattr(atlas, 'region_coords') else plotting.find_parcellation_cut_coordinate

plotting.plot_connectome(
    correlation_matrix,
    coords,
    edge_threshold="95%",  # Muestra solo el 5% de las conexiones más fuertes
    title="Conectoma Funcional (Top 5% conexiones)",
    display_mode='ortho'
)
plotting.show()
```

Detalles técnicos clave

- **Filtro temporal ($0.01 - 0.1$ Hz):** Elimina ruido de baja frecuencia (deriva térmica del resonador) y alta frecuencia (frecuencia cardíaca y respiratoria).
- **Confusos (Regresión de ruido):** Incluye los 6 parámetros de movimiento de cabeza (X, Y, Z, cabeceo, cabeceo corto, guiñada), además de las señales medias de la materia blanca y el líquido cefalorraquídeo.
- **Detrending y Estandarización:** El *detrending* elimina cambios lineales lentos en el tiempo y la estandarización escala la señal a Z-scores (media 0, desviación estándar 1).
